Understanding RealTOA

A Telegram Bot that automates the mileage tracking system for SAF Transport Operators

1 CONTENTS

2 Introduction 3
2.1 Context 3
2.2 Current Workflow 3
2.3 Objectives 4
3 Ideation 5
3.1 Design Constraints 5
3.2 Proposal 1 – integrating forms into telegram 6
3.2.1 Prototype of Proposal 1 8
3.3 Proposal 2 – background, parsing telegram bot 10
3.3.1 Prototype of Proposal 2 12
4 Testing 14

2 INTRODUCTION

2.1 CONTEXT

In the Singapore Armed Forces (SAF), Transport Operators (TOs) are required to log the mileage accumulated during each mission. This record is important because upon reaching 4000km, a TO is eligible to convert his military driving license to a civilian one, free of cost.

Therefore, **accurate, timely and complete mileage records of TOs is important**.

2.2 CURRENT WORKFLOW

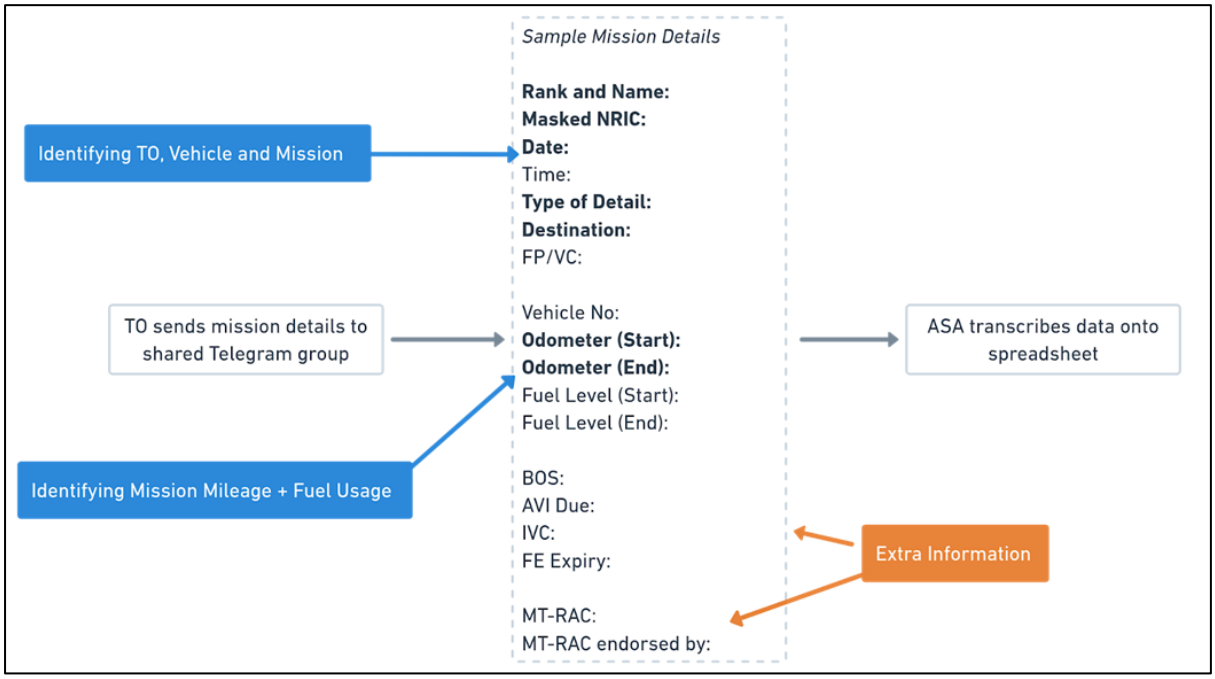


Figure 1: Current Workflow

Currently, the TO:

1. Uploads mission information into a shared Telegram group
2. Admin Support Assistants (ASAs) then transcribes information onto a spreadsheet

From this workflow, me and fellow other TOs have noticed some potential and ongoing failure points, observed in Table 1 below.

Failure	Impact
Messages missed or overlooked by ASA	Entry is not recorded → lost mileage for mission
Incorrect mission details sent	Inaccurate odometer logs → inaccurate mileage totals (worse if it is lower than actual)
No TO visibility of spreadsheet	TOs cannot verify their own records and mileage without requesting the ASAs for access
Numerous TOs → numerous mission details sent everyday	Potential delays in transcription + repetitive, tiresome for ASAs.

Table 1: Identified Failure Points

2.3 Objectives

Hence, this project aims to:

- **Reduce Human Error** in mileage recording for both the TOs and ASAs.
- **Improve the efficiency and accuracy of updating** and maintaining the mileage records
- Create a more **transparent and reliable** mileage recording system
- **Automate the mileage recording system**

3 IDEATION

3.1 DESIGN CONSTRAINTS

My first idea to automate the current workflow was to use Google Forms, due to its easy, straightforward integration with Google Sheets for automatic data recording. However, observations in the node show that TOs are **less responsive** when required to submit information through any type of forms. This is mainly due to 2 reasons.

Firstly, the form-based submission requires TOs to **fill up multiple fields,** making it 'appear' **slow and tedious**. Currently, TOs are used to sending information quickly through Telegram, often by copying and modifying mission details sent by other TOs. Introducing a new, structured form disrupts this established workflow, and increases the effort required to submit information.

Secondly, Google Forms does not provide any feedback to the TO, regarding if their information has been appended to the spreadsheet, unless they manually open and check the spreadsheet. This reduces confidence and can discourage adoption.

Thus, the following guidelines have been established, as observed in Table 2 below.

Problems Identified	Design Guidelines
TOs are not responsive to Google Forms	The solution should **integrate directly** with TOs' existing telegram group
	The solution should **require minimal changes to the current message-posting behaviour.**
	The solution should **not require** TOs to use **any additional software or external tools**.
TOs do not have immediate confirmation if their information has been appended to the spreadsheet.	The solution should **notify TOs** once their information has been appended in the spreadsheet.

Table 2: Design Guidelines

3.2 PROPOSAL 1 – INTEGRATING FORMS INTO TELEGRAM

To address the constraints identified in 3.1 (Page 5), Proposal 1 involves creating a Telegram bot that **replicates the structured input workflow of Google Forms.**

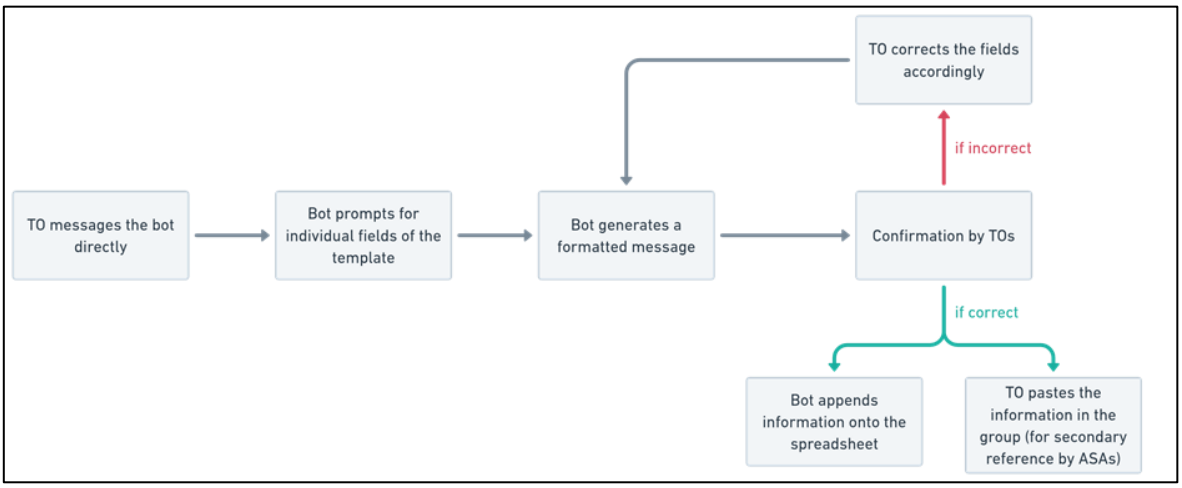


Figure 2: Proposal 1's Workflow

The workflow operates in the following manner:

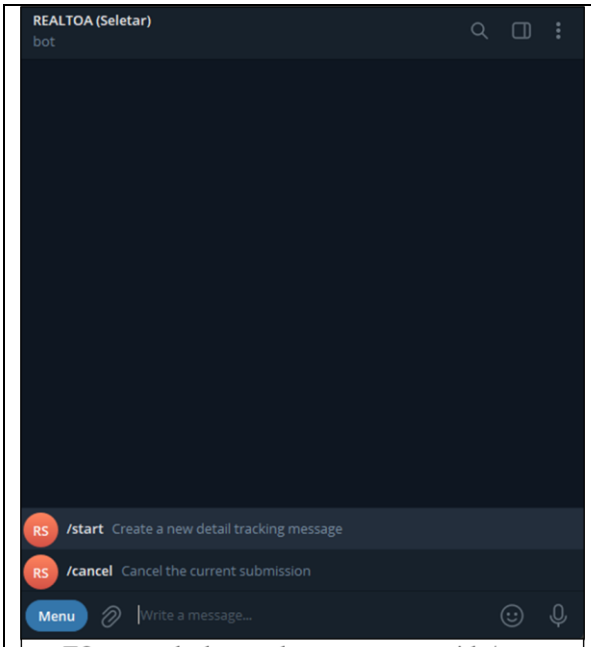
1. The TO initiates the process by messaging the bot
2. The bot sequentially prompts the TOs for each field in the mission template (shown in Figure 1: Current Workflow), similar to completing a Google form. A key benefit is that **information regarding TO's personal details are pre-filled.**
3. The bot generates a formatted message, containing all completed fields.
    a. Upon checking and confirmation by the TO, this information is appended to the spreadsheet, where it is verified by the ASAs.
    b. The TO sends this message in the Telegram group (for secondary reference by ASAs).

When creating Proposal 1, these strengths and weaknesses were observed, which can be helpful in designing more proposals.

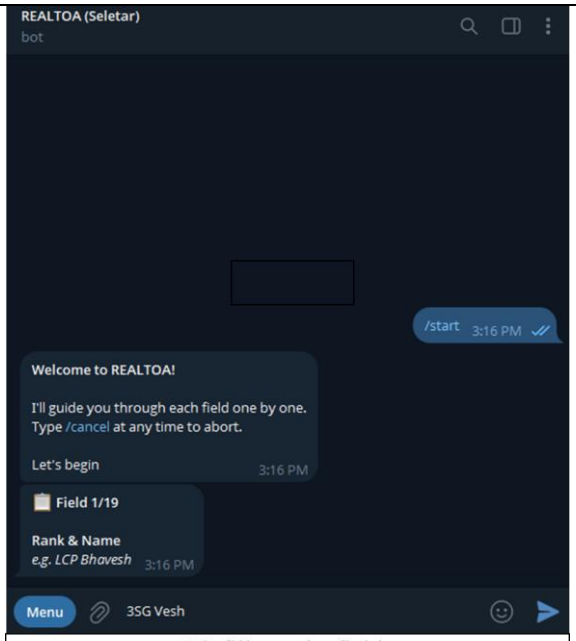
Strengths	Weaknesses
Pre-filled fields → TOs send mission details faster	Numerous prompts still yet to answer → may perceive bot to be annoying
No additional software required (everything native in Telegram) → less behavioural friction → higher likelihood of compliance	Introduction of a bot may still involve some behavioural friction
Confirmation Loop → TOs forced to verify their records → Accurate mileage tracking	Copy, pasting from the bot to the telegram group can be mundane → skipped → secondary reference for ASAs missing
Logic for appending data to spreadsheet is simple and straightforward.	Bot needs to stay online 24/7 for constant messaging
	API costs for appending to spreadsheet

Table 3: Initial Evaluation of Proposal 1

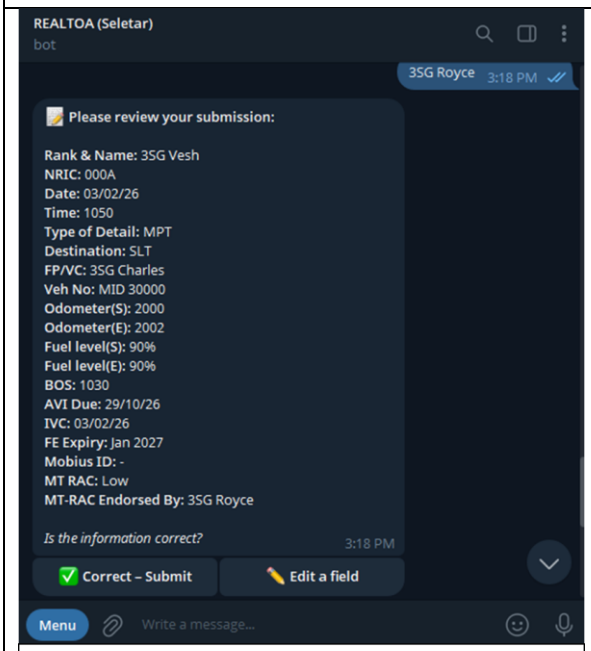
3.2.1 Prototype of Proposal 1



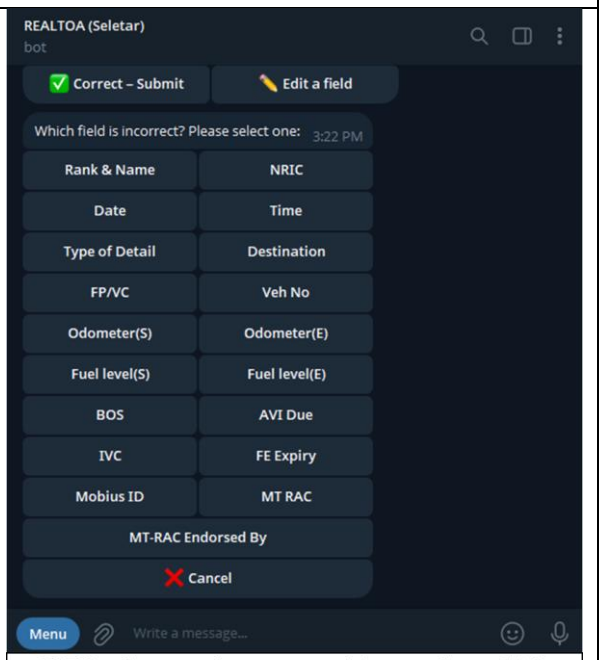
TO opens the bot, and starts a convo with /start



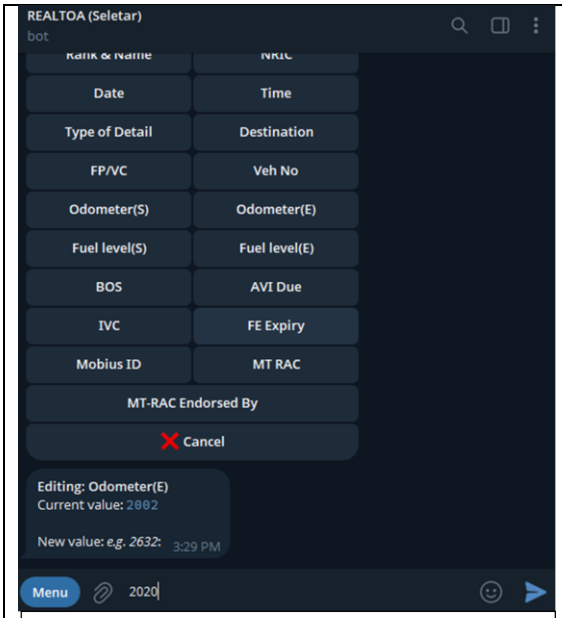
TO fills up the fields



After finishing the fields, the bot prompts TO to check sent information thus far

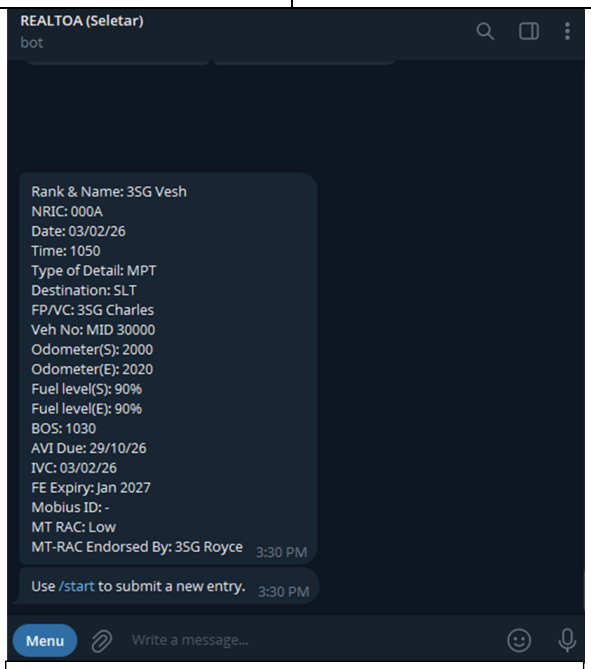


If TO selects no, bot prompts him to select which field to update.



Bot shows what field is being edited, its current value, and proposed format of the new value.

Bot sends the updated message for confirmation.



If the TO selects yes, this message is sent for the purpose of 'copy-pasting' to the Telegram group with ASAs

Table 4: Screenshots of Proposal 1

3.3 PROPOSAL 2 – BACKGROUND, PARSING TELEGRAM BOT

Proposal 2 is also a telegram bot, but with different functionality and has been created to minimise weaknesses experienced in Proposal 1 (Table 3). This bot is responsible for reading the mission details sent and extracts information to append to the spreadsheet, while maintaining a similar confirmation loop as proposal 1.

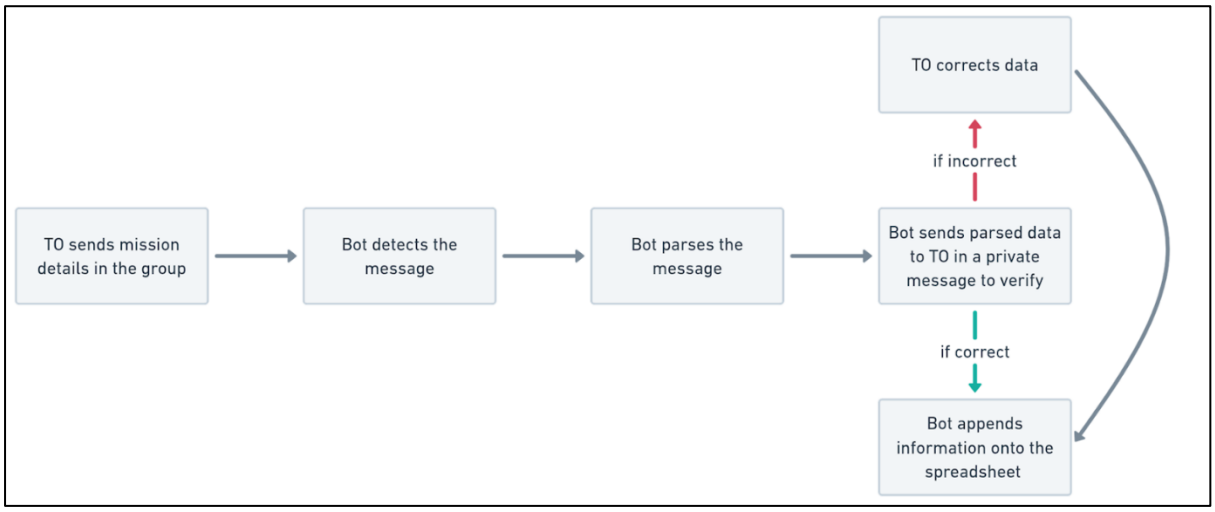


Figure 3: Proposal 2's Workflow

The workflow operates in the following manner:

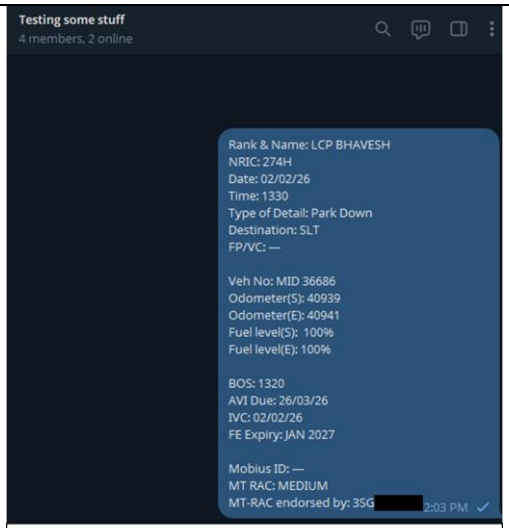
1. The TO sends mission details in the Telegram group normally.
2. The bot detects the message.
3. The bot extracts key information from the message using *regex parsing*.
4. The bot messages the sender TO privately to confirm extracted information.
    a. If accurate, this information is appended to spreadsheet.
    b. If inaccurate, the TO is prompted to correct relevant fields in the bot, and is expected to edit information in the Telegram group.

These strengths and weaknesses were observed, when creating Proposal 2.

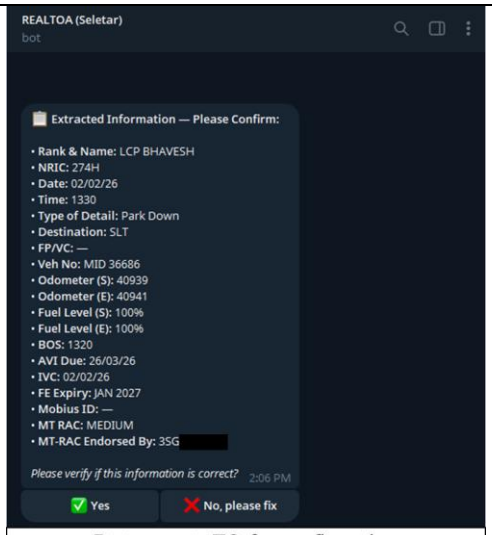
Strengths (proposal 2)	Weaknesses (proposal 1)	Weaknesses (proposal 2)
Minimised interaction with the bot, the bot prompts rather than the TO prompts	~~Introduction of a bot may still involve some behavioural friction~~	Regex Parsing logic may be difficult to code and implement due to numerous, possible edge cases.
Less prompts and questions by the bot	~~Numerous prompts still yet to answer → may perceive bot to be annoying~~	
Less work done by TOs (information remains in the telegram group primarily)	~~Copy, pasting from the bot to the telegram group can be mundane → skipped → secondary reference for ASAs missing~~	
	Bot needs to stay online 24/7 for constant messaging	
	API costs for appending to spreadsheet	

Table 5: Initial Evaluation of Proposal 2

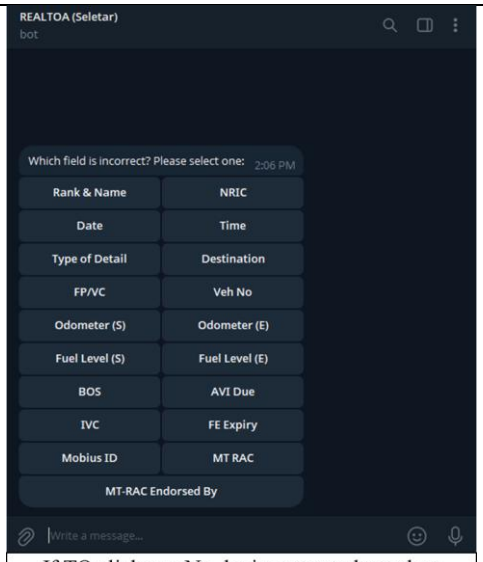
3.3.1 Prototype of Proposal 2



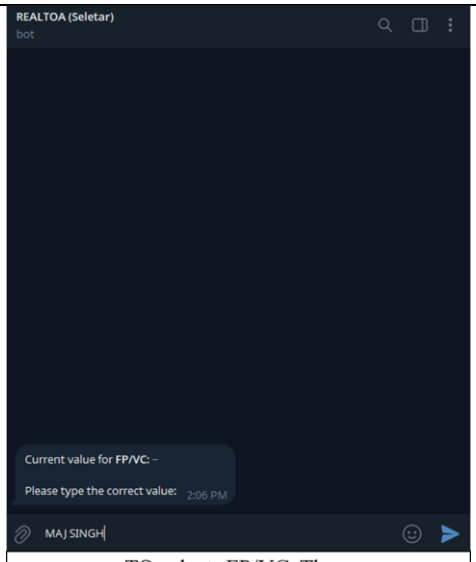
TO sends mission details in the group



Bot requests TO for confirmation

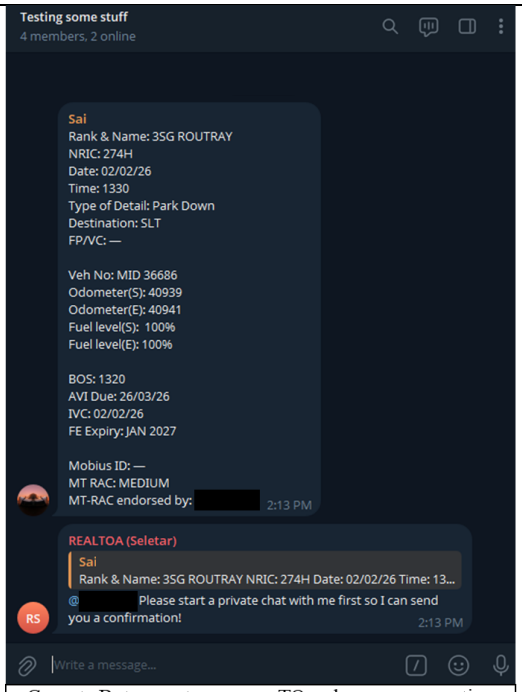


If TO clicks on No, he is prompted to select which field.



TO selects FP/VC. Then,
TO is prompted to update this field.

Message updates. TO prompted to confirm message again.



Caveat: Bot cannot message TO unless a conversation has started.
Hence, bot prompts TO in the group to start a convo.

Table 6: Screenshots of Proposal 2

4 TESTING

To test the proposals, there are 2 main categories to consider:

1. Behavioural Testing (determining if the TOs like the solution)
2. Technical Testing (determining if the TOs can handle the expected load and function reliably)

The testing will be conducted in the following manner:

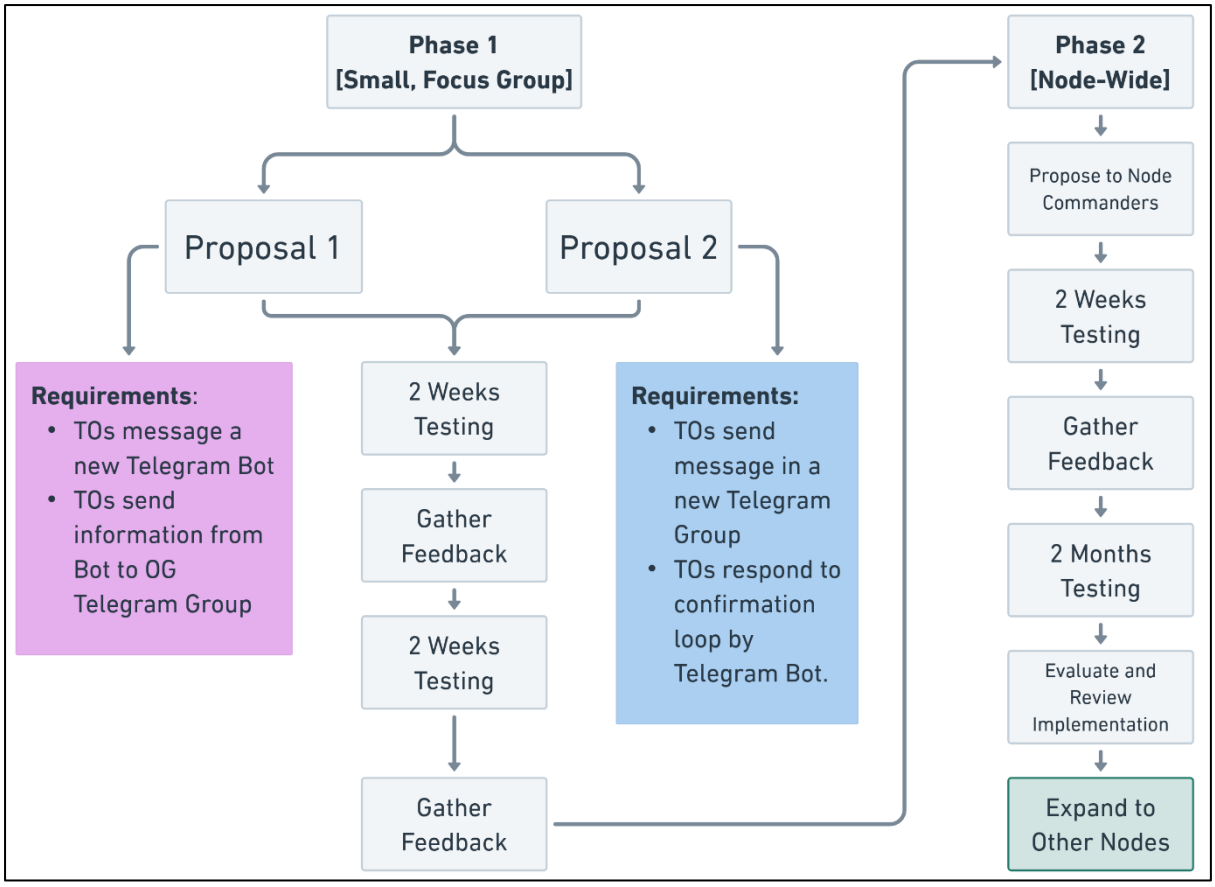


Figure 4: Proposed Testing Flowchart

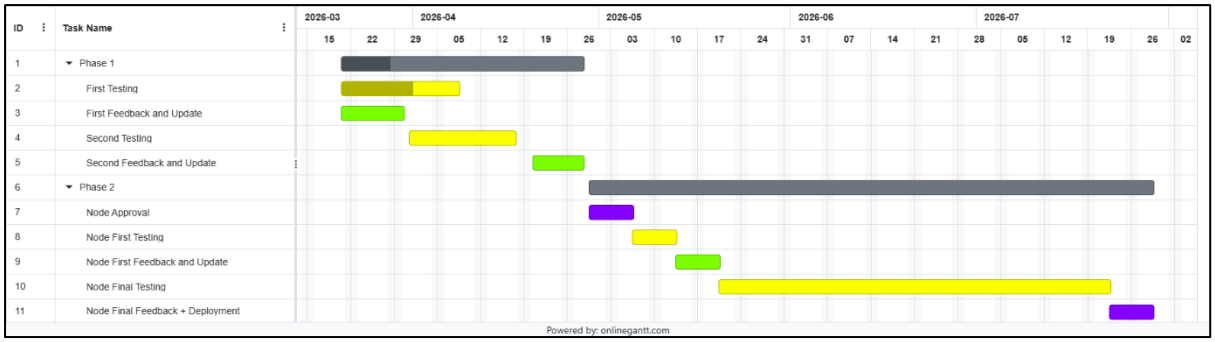


Figure 5: Gantt Chart for Testing

Task Name		Start Date	End Date
Phase 1	First Testing	02/03/2026	16/03/2026
	First Feedback and Update	17/03/2026	22/03/2026
	Second Testing	23/03/2026	06/04/2026
	Second Feedback and Update	07/04/2026	12/04/2026
Phase 2	Node Approval	13/04/2026	26/04/2026
	Node First Testing	27/04/2026	11/05/2026
	Node First Feedback and Update	12/05/2026	17/05/2026
	Node Final Testing	18/05/2026	17/07/2026
	Node Final Update + Deployment	18/07/2026	27/07/2026

Table 7: Important Deadlines